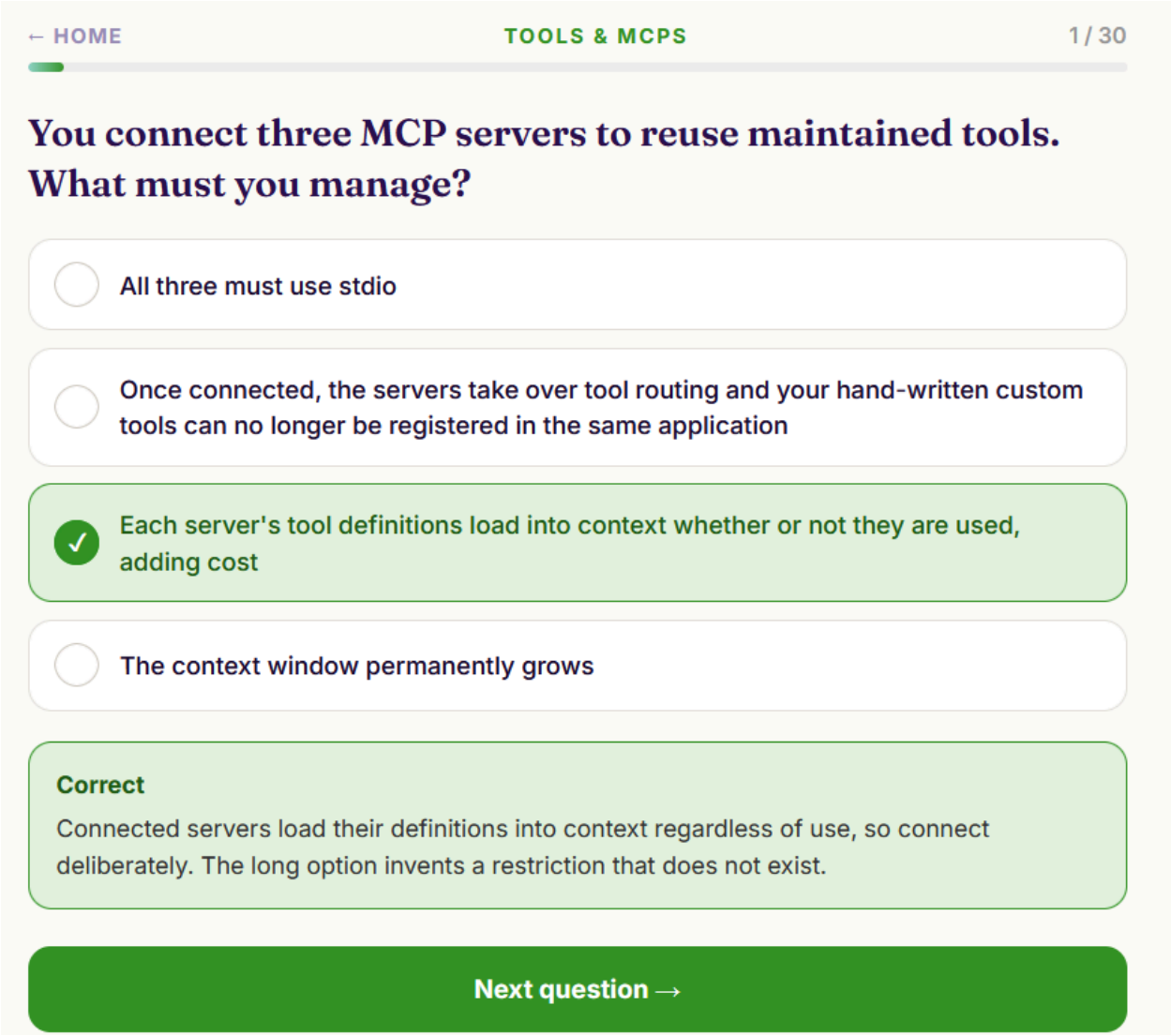
https://claude.ai/public/artifacts/83eee27a-0509-45c1-be08-cc3e8b4cbfc8

Set 1 :



A regulated client requires that data never leave their existing cloud. How should this drive platform choice?

- ✓ Run on the provider (Bedrock or Vertex) that keeps data inside the client's cloud and compliance boundary
- Only local self-hosting can ever be compliant
- Default to the first-party API, since it is always cheapest and gets new models first, then layer network controls on top to satisfy the auditors
- Pick whichever platform has the newest model

Correct

When data residency governs, choose the platform that keeps data in the client's cloud and compliance boundary. Cost, model recency, and an assumption that only local hosting is compliant all miss the constraint.

Next question →

Prompt-only formatting keeps failing on untested inputs that break your parser. Best next step?

- ✓ Constrain output in the API: JSON schema for the response and strict tool use for arguments
- ○ Switch to a bigger model
- ○ Expand the prompt with an exhaustive set of formatting rules and several reminders, then retry any input whose output fails to parse until it eventually conforms to the schema
- ○ Lower temperature to 0 and trust the output

Correct

Move control into the API with structured outputs and strict tool use. The long option is more of the prompt-only approach that already failed; temperature and model size do not enforce structure.

Next question →

Two runs of the same prompt at temperature 0 return slightly different wording. Which explanation is correct?

- This is impossible at temperature 0, so one of the two calls must have silently used a different model version or returned a cached response from an earlier request
- Streaming was left on for one of the calls
- ✓ Sampling can still introduce variation; temperature 0 favours the likeliest tokens but does not hard-guarantee identical output
- The two calls used different context windows

Correct

Even at temperature 0, determinism is not guaranteed. The long option asserts a false impossibility; streaming and window size do not explain wording variation.

Next question →

Which TWO statements about prompting modes are accurate? (Pick 2)

Select 2 · chosen 2

- ✗ Multi-shot permanently fine-tunes the model on the examples
- ✓ Multi-shot examples guide the output on that call, at extra token cost
- ☐ Zero-shot always outperforms few-shot once the task is well-specified
- ✓ Single-shot includes exactly one worked example

Not quite

Single-shot is one example; multi-shot adds several to shape output per call at token cost. Examples do not fine-tune the model, and zero-shot is not universally superior once specified.

Next question →

An MCP connection trace shows 401 on both the first attempt and the retry, with the credential read as a plaintext value from a file at a known path. Correct fix?

- Switch this service from API-key auth to OAuth
- Rotate the key and write the new value back into the same credentials file, since a fresh key is what the service will accept
- ✓ Rotate the rejected key so the connection can authenticate, and move the credential out of the file into a runtime environment variable so it is never stored in plaintext again
- Add a retry with backoff so a third attempt can succeed

Correct

Two problems are stacked: a rejected key and an insecurely stored one, so both must be fixed. Rotating back into the same file leaves the storage defect, OAuth does not match a service-account identity, and backoff does not fix a rejected key.

Match each prompt failure to the missing technique. Which TWO pairings are correct? (Pick 2)

Select 2 · chosen 2

- ✓ Drift across turns → tighten and complete the system prompt
- ✓ Output in the wrong shape → add an output constraint
- ✗ Wrong shape → add more few-shot examples of reasoning
- ☐ Hallucinated structure → raise the temperature

Not quite

Wrong shape signals a missing output constraint; drift signals an underspecified system prompt. A hallucinated structure needs few-shot examples (not temperature), and shape is not fixed by reasoning examples.

Next question →

When should a streamed turn be written into conversation history?

- ✓ Only after message_stop
- Once the first content_block_start event arrives, so the stored history stays current with the model in real time
- Whenever the socket closes for any reason
- After the first token

Correct

A stream ending is not a message completing: commit only after message_stop, discarding partial turns on interruption. Committing early risks storing a half-built turn.

Next question →

Across a multi-turn exchange using extended thinking, what must happen to thinking blocks?

- They should be deleted before the next call
- ✓ They must be returned unchanged
- They belong in the system prompt
- They should be summarised and re-sent so history stays compact while preserving the reasoning for later turns

Correct

Thinking blocks must be returned unchanged or the next request fails. Summarising, deleting, or relocating them breaks the exchange.

Next question →

You're wiring an agent whose tool can irreversibly modify a customer system. Where does the human checkpoint belong?

- Only in the retry handler
- After the first production write, added once you have watched the agent behave on real traffic and can place the gate precisely
- It can be skipped if tests passed
- ✓ In the design, gating the irreversible action before the loop is built

Correct

For irreversible actions the gate goes into the design before the loop is wired. Waiting until after the first write, or only on retry, lets the irreversible action through first.

Next question →

Two tools are both described as retrieving information and Claude often calls the wrong one. Best single fix?

- Give each tool a richer, strongly-typed input schema with distinctive parameter names so the model has more signal to tell them apart
- ✓ Add to each description a clear statement of when NOT to use it
- Delete one of the tools
- Increase the model tier

Correct

Claude matches on the description, so an exclusion condition resolves most wrong-tool bugs. The long option is the trap: differing input schemas do not help when descriptions overlap.

Next question →

Before writing multimodal ingestion, you estimate image cost. Which statement is accurate?

- Images are billed only on output tokens
- All images cost a flat token amount regardless of size or model
- ✓ Visual tokens scale with image dimensions and the per-image ceiling varies by model tier, so a high-resolution original can cost many times a thumbnail
- Describing images in text is always cheaper than sending them

Correct

Image cost grows with dimensions and the ceiling differs by tier, so resolution matters a lot. Flat pricing and output-only billing are wrong, and text description is not universally cheaper.

Next question →

A PreToolUse hook must block a disallowed read. Which TWO are true? (Pick 2)

Select 2 · chosen 2

- [x] Exiting with code 2 blocks the call, and text written to stderr becomes the message Claude sees
- [ ] PostToolUse could also block, by inspecting the result and undoing the read
- [x] It must use PreToolUse, because that fires before the tool executes
- [ ] Exiting 0 with the reason on stdout is what signals the block

Correct

Only PreToolUse can block, and the hook signals a block with exit code 2 plus a reason on stderr. PostToolUse runs after the tool, so it cannot prevent the read, and exit 0 allows the call.

Next question →

A multi-turn session grows until a request exceeds the window mid-generation. What happens?

- The window auto-expands for that call
- ✓ The output is truncated and the response carries a context-window-exceeded stop reason
- The call returns a 200 with empty content
- The request is silently accepted and the model drops whichever earlier turns it judges least relevant to make room for the rest

Correct

Overflowing mid-generation yields truncated output with a model_context_window_exceeded stop reason. The window is fixed: it neither silently drops turns nor expands.

Next question →

When should hard cost and latency budgets be set?

- Only once costs exceed forecast
- ✓ Before the architecture is decided
- After the system is built and profiled under real traffic, when you finally have accurate numbers to base the ceilings on
- At the first incident review

Correct

Set hard budgets before architecture so you can check the design against them before writing code. Waiting until after the build, an overrun, or an incident means the budget never shaped the design.

Next question →

A scheduled headless job uses the Agent SDK and expects the Skill to load from the repo. What must be configured?

- Place SKILL.md in .claude/skills and rely on the terminal default
- ✓ Enable filesystem sources by setting settingSources explicitly so the agent loads skills from the project, rather than relying on a default, and confirm the current default against the Agent SDK reference
- Set the managed-agents beta header
- Send the code-execution and skills beta headers

Correct

For an Agent SDK job that must load skills from the repo, set settingSources explicitly rather than trusting a default. The terminal placement, managed-agents header, and code-execution headers belong to other runtimes.

You must grade thousands of open-ended summaries where exact-match won't work. Sound approach?

- Eyeball a handful and extrapolate
- Exact-string match to a reference
- Trust each summary's own stated confidence and grade on that, since the model has the most context on whether it succeeded
- ✓ An LLM-as-judge scoring against explicit criteria, validated against a human-labelled sample

Correct

Open-ended grading at scale uses an LLM judge against explicit criteria, calibrated to human labels. Self-reported confidence is unreliable, exact-match fails on free text, and a handful of samples does not generalise.

A developer wants a review-checklist Skill to load when they ask for a review in the Claude Code terminal. What must be configured?

- ✓ Place SKILL.md in .claude/skills with a description that matches review requests
- Set settingSources explicitly for the Agent SDK
- Send the code-execution and skills beta headers on every request
- Define the agent as an API resource that lists the skill and set the managed-agents beta header on the calls, writing the skill so its steps do not depend on any local files

Correct

In the Claude Code terminal, a Skill loads from .claude/skills when its description matches the request. The long option is the Anthropic-hosted-agent setup; the others belong to the Messages API and Agent SDK runtimes.

A SKILL.md runs on the author's machine but breaks when a teammate clones the repo, because step 1 calls /Users/alexmorgan/projects/deploy-utils/validate.sh. Correct fix?

- Remove the step so the skill no longer calls an external script
- Replace it with another absolute path that points to a shared network drive everyone can reach
- ✓ Reference the script from the project root via CLAUDE_PROJECT_DIR so it resolves wherever the repo is cloned
- Use a home-directory shortcut like ~/projects/deploy-utils/validate.sh

Correct

The defect is the absolute path to the author's home directory; referencing from CLAUDE_PROJECT_DIR resolves from the repo root anywhere. Another absolute path or a home shortcut just moves the same problem, and removing the step drops the capability.

Which TWO success criteria are specific enough to build an eval against? (Pick 2)

Select 2 · chosen 2

- [ ] A helpful, high-quality summary of the thread
- [x] A refund decision of exactly 'approve' or 'deny' with a one-line reason
- [ ] A response that satisfies the user
- [x] A two-sentence summary that lists every action item and its owner

Correct

Gradeable criteria state a checkable output. 'Every action item and its owner' and a constrained 'approve/deny plus reason' can be graded; 'helpful' and 'satisfies the user' are too vague to check.

Next question →

Configuring Claude Code for a trusted refactor that must auto-approve edits, block destructive shell commands, and never read .env.production. Which TWO settings pieces are correct? (Pick 2)

Select 2 · chosen 2

- [ ] allow ["Bash(*)", "Edit(*)"] with no destructive-command gate
- [ ] defaultMode "bypassPermissions"
- [x] deny ["Read(.env.production)"]
- [x] allow ["Bash(npm run:*)"], deny ["Bash(rm:*)", "Bash(git push:*)"]

Correct

Allowing safe commands while denying destructive ones gates shell execution, and denying Read(.env.production) enforces the path restriction at the settings layer. bypassPermissions removes the guard, and an unrestricted allow is the opposite of the requirement.

Your settings auto-approve edits. Mid-refactor the agent proposes editing a deployment-config file that several production services read. Where should a human gate sit for this one action?

- ✓ A human reviews and approves this specific change before the write executes, because a wrong value is hard to undo and reaches systems beyond the file
- Add bypassPermissions so the agent never pauses
- Nowhere; the settings already auto-approve edits
- Review it after the write, in the next pull request

Correct

Auto-approving edits is fine as a default, but the worst-case high-cost action still warrants a human gate before the write. Reviewing after the fact is too late for a change that reaches production services.

An agent returns a wrong answer. What most directly isolates whether the tool, the model, or the orchestration failed?

- Swapping in a bigger model
- Re-reading the final answer closely to infer from its wording where the reasoning must have gone wrong
- Rerunning it a few times
- ✓ A trace of the run

Correct

A step-by-step trace shows where the run diverged, isolating tool versus model versus orchestration. Re-reading, rerunning, and model swaps are guesses that do not localise the fault.

Preparing a build for reuse and safe deployment. Which TWO practices matter most? (Pick 2)

Select 2 · chosen 2

- [ ] Keep prompts inline and unversioned to reduce overhead
- [x] Pin the model version in production
- [x] Version your prompts so a change can be attributed and rolled back
- [ ] Always deploy on the newest model automatically to stay current

Correct

Pinning the model and versioning prompts make changes deliberate, attributable, and reversible. Auto-upgrading and unversioned inline prompts remove exactly the control that reuse and rollback depend on.

Next question →

A security-scanning MCP server must be deployed to every developer's Claude Code installation across the org. Which transport and scope fit?

- ✓ HTTP + Enterprise (managed settings)
- stdio or HTTP + Local
- stdio + Local, installed once on each machine and shared informally so each developer keeps control of their own copy
- HTTP + Project (.mcp.json)

Correct

An org-wide mandate for every install is an enterprise-scope, HTTP-transport deployment through managed settings. Local scopes are per-developer, and project scope covers one team's repo, not the whole organisation.

A large, stable system prompt is sent on every call in a high-volume app. What does prompt caching do, and what is its limit?

- It makes all later calls free regardless of what changes in the prompt
- It reduces output-token cost specifically
- ✓ It reuses the stable prefix at reduced cost, but the cache expires and must be refreshed, and only the unchanged prefix benefits
- It only works in batch mode

Correct

Caching cuts the cost of re-sending an unchanged prefix, but caches expire and only the stable portion is cached. It does not make calls free, target output tokens, or require batch.

Next question →

You have many trivial calls and occasional hard ones, and want to pay for deep reasoning only when it helps. Which fits?

- Extended thinking pinned to maximum effort on every call, so quality is never at risk on the genuinely hard ones
- ✓ Adaptive thinking, with effort scaled to the task
- Fast mode on every call
- A larger model tier for all calls

Correct

Adaptive thinking spends reasoning effort only where it changes the answer. Always-max overpays on trivial calls, fast-mode-everywhere underserves the hard ones, and a bigger tier does not target the mix.

Next question

In the failure-handling section of a design doc, what is the core task?

- Raise the request timeout
- ✓ Enumerate the errors production will throw, mark each retriable or terminal, and define the user-facing outcome when recovery fails
- Wrap every external call in an automatic retry loop that keeps trying until it succeeds, since most production failures are transient and clear on their own within a few attempts
- Switch to a larger model for resilience

Correct

Failure handling means naming each error, classifying it retriable or terminal, and stating the fallback for the user. Blind retry loops, longer timeouts, and bigger models are not an error path.

Next question →

Naming the trust boundary on paper turns least privilege into something you can...?

- ✓ Enforce with a hook, rather than a setting you remember to add later
- Skip, if the model is well-behaved
- Defer entirely to the security team
- Guarantee automatically, because documenting the boundary is itself what applies the restriction at runtime across every tool the agent can reach

Correct

A named boundary becomes an enforceable design decision, implemented with a hook. Documentation alone enforces nothing at runtime, and least privilege cannot be skipped or wholly delegated away.

Next question

Production sessions turn out short and numerous, unlike the long dev sessions, and in-context memory now fails early. Which model most likely fits?

- Summarised in-context memory
- Keep in-context memory but raise max_tokens each session so history has more room to accumulate before it fails
- Stateless for every session
- ✓ External storage that persists state across the many short sessions

Correct

State must survive across many short sessions, which external storage provides. Raising max_tokens does not survive session boundaries, stateless drops needed continuity, and summarised memory is still in-context.